WIMBL

Autonomous sports-travel agents that learn from every trip

THE CORE PROMISE
Give WIMBL a travel goal. It researches, plans, verifies, and improves future runs with minimal interruptions. It asks the traveler only when a genuinely important decision cannot be resolved safely on its own.

Build brief for engineering agents

Product scope • agent architecture • Version N+1 learning • low-interruption autonomy • NVIDIA and Crusoe stack

Implementation brief. Validate live model availability, endpoint compatibility, and benchmark performance before final selection.

1. Product definition

WIMBL is an autonomous sports-travel planning system. A user expresses an interest or goal, such as attending a major football match in Europe. WIMBL discovers events and travel options, assembles a feasible end-to-end package, verifies key claims, and presents ranked recommendations with evidence and clear uncertainty.

The differentiator is not simply that WIMBL uses multiple agents or stores memory. Its agent family learns from completed trips, tests proposed improvements against a baseline and unseen scenarios, and only promotes changes that demonstrate measurable gains.

Product principles

- **Autonomy by default:** research and plan without asking a long sequence of questions.
- **Evidence before confidence:** material recommendations trace to a source URL, user-provided fact, or clearly labeled assumption.
- **Done means done:** define acceptance criteria, independently verify them, and do not declare success based only on the planner's claim.
- **Shared brain:** agents share scoped, provenance-backed lessons and trip findings.
- **Version N+1:** reflect, propose, evaluate, promote or reject, and retain rollback capability.
- **Safe autonomy:** research and prepare independently; require authorization before purchases, bookings, or other consequential external actions.

Primary user journey

1. User gives a natural-language goal and any known constraints.
2. Controller extracts hard constraints, infers safe defaults for missing preferences, and freezes acceptance criteria for the run.
3. Agent family researches events, tickets, transport, lodging, and local logistics.
4. Planner creates ranked packages and calculates a transparent total-cost estimate.
5. Independent verifier checks feasibility, dates, source freshness, ticket legitimacy signals, transfer timing, and unresolved risks.
6. Controller repairs or replans within limits; the user is interrupted only if a genuinely blocking decision remains.
7. After completion, the learning engine evaluates outcomes and proposes improvements for future runs.

Non-goals and honesty rules

- Do not imply a price or inventory is live unless the source supports the claim and the observation time is recorded.
- Do not claim a reservation, ticket, or booking is confirmed unless a real booking flow confirms it.
- Do not let learned lessons weaken explicit user constraints, frozen acceptance criteria, or safety requirements.
- Do not invent benchmark wins. Show measured results or clearly label the benchmark as not yet run.

2. Agent architecture

Use a controller-led system with specialist agents, an independent verifier, and a separate learning/promotion pipeline. Prefer short-lived trip children for task execution and persistent, versioned parent agents or registries for durable knowledge.

Component	Responsibility	Key outputs
Trip Controller	Owns task lifecycle, acceptance criteria, budgets, retries, replanning, stopping, and rare escalation.	Run state, assumptions, completion status, blocker reason.
Sports Parent	General travel knowledge, user preference rules, verification standards, reusable strategies.	Scoped inherited lessons and policy defaults.
Football Parent	Football-specific discovery strategy, ticket-sale patterns, match-day timing, trusted source guidance.	Football lessons with provenance and validation history.
Trip children	Specialists for tickets, flights, hotels, local transit, price watch, and coordination.	Findings, options, citations, assumptions, risks.
Sibling coordination	Share relevant trip findings in near real time, e.g. arrival time affects hotel dates and airport transfer.	Shared trip facts with source and timestamp.
Independent verifier	Checks package against frozen criteria; can reject completion and request repair.	Pass/fail checks, defects, evidence, residual risk.
Learning engine	Retrospective, change proposal, benchmark and held-out evaluation, promotion/rejection, rollback.	Versioned candidate, evaluation report, decision record.
Shared memory	Stores lessons by scope, provenance, confidence, recency, and validation status.	Searchable lesson registry and lineage.

Suggested lesson record

```
lesson_id, lesson, from_agent, scope, lineage, weight, evidence_urls, source_type, confidence,
trips_validated, successes, failures, last_validated, expires_at, version_introduced, evaluation_status
```

Example: “Check official club ticketing and resale pages before third-party listings.” Evidence should include the official source, scenarios tested, and whether it reduced risk or improved the result. User preferences retain user-origin provenance and are not internet-derived facts.

Lesson influence should combine lineage relevance, smoothed observed success, independent validation count, and recency. Avoid unbounded influence from raw trip counts. Hard user constraints and safety rules are not ordinary weighted lessons.

3. Version N+1: the learning engine

The improvement loop is a first-class feature. It should run after a trip or on a controlled evaluation schedule, without automatically rewriting production prompts or code.

1. Observe

Collect outcomes: verified defects, stale data, failed tools, cost, latency, user corrections, and useful decisions. Keep observations separate from conclusions.

2. Reflect

Generate specific, testable explanations. Example: checking official ticket sources earlier reduces unreliable listings.

3. Propose

Create one isolated candidate change to a lesson, prompt, tool policy, routing rule, or agent workflow. State the hypothesis and predicted effect.

4. Establish baseline

Run the current version on fixed benchmark scenarios. Record package quality, feasibility, verification errors, cost, and runtime.

5. Evaluate candidate

Run the candidate on the same scenarios and on held-out scenarios it was not tuned against. Repeat stochastic runs where useful.

6. Decide

Promote only if safety and correctness checks pass and target metrics improve beyond a predefined threshold. Otherwise reject or revise.

7. Roll out and monitor

Version the change, record the promotion rationale, monitor for regressions, and restore the prior version if required.

8. Teach the family

Propagate proven lessons to the appropriate parent or scope. Preserve attribution so a child can explain which inherited lesson influenced a decision.

Promotion gate

Do not promote a candidate merely because one trip looked successful. Require evidence provenance, baseline comparison, held-out tests, no critical regression, and an auditable decision. Define trade-offs before evaluation because one metric may improve while another worsens.

Recommended metrics

- **Verified package success:** share of runs passing all required checks.
- **False-completion rate:** runs marked complete despite a failed acceptance criterion.
- **Generalization:** candidate improvement on held-out trip scenarios.
- **Cost per verified success:** model and tool cost per successful package.
- **Time to verified result:** end-to-end duration.

- **Regression rate:** promoted changes that worsen subsequent performance.
- **Inheritance attribution:** decisions linked to the lesson or version that influenced them.
- **Provenance coverage:** material claims and lessons traceable to evidence.

4. User questions should be rare

WIMBL should default to autonomous progress. Missing information is not automatically a reason to ask: first infer a reversible default, search for the answer, compare options, or disclose an assumption.

Question policy

- **Default to zero clarification questions.** Start researching with available information.
- **Infer reversible preferences.** If no hotel style is specified, show sensible options rather than blocking the run.
- **Research unknown facts.** Use sources to resolve schedules, venue locations, transit, and published ticket policies.
- **Offer ranked alternatives.** Compare plausible interpretations against inferred goals and explain trade-offs.
- **Ask one focused question only when blocked.** State the exact decision that cannot be resolved and why it matters.
- **Require approval for consequential actions.** Do not purchase, book, or commit money without necessary authorization.

Examples

Situation	Preferred behavior
No hotel preference	Compare practical, well-reviewed options in several price bands. Do not ask first.
No departure airport	Use reliable profile context if available; otherwise ask only if it materially changes the result. Label assumptions.
Several matches fit	Research and rank by total cost, schedule, travel burden, and match appeal.
Required booking choice missing	Prepare the recommendation and ask for required authorization before committing.
Sources conflict or fail	Try authoritative alternatives, lower confidence, and report unresolved issues. Ask only if blocked.

Track interruptions per completed trip, completion without intervention, and whether questions were truly necessary. Do not optimize for zero questions at the expense of safety or correctness.

5. NVIDIA and Crusoe stack recommendation

Model selection is separate from agent runtime and sandboxing. These layers can be combined if the actual endpoints, frameworks, and hackathon rules are compatible.

Layer	Recommended role	Decision notes
Crusoe Managed Inference	Required inference provider for model calls, based on the project requirements.	Choose a model actually exposed by the current Crusoe catalog. Verify API compatibility, context limit, rate limits, pricing, and tool calling.
NVIDIA Nemotron	Preferred model family to investigate for agentic reasoning, planning, and tool use.	Use a specific variant only if available through Crusoe or an event-approved deployment route. Benchmark on WIMBL tasks.
NVIDIA NemoClaw / OpenShell	Potential agent deployment and sandbox layer for always-on agents.	Check network, filesystem, process, and inference guardrails, framework support, endpoint compatibility, and host-side secret handling.
You.com	Web discovery adapter for current event and travel research.	Store source URLs, observation time, extracted claim, and freshness metadata. Search results do not automatically prove availability.
Independent evaluator	Benchmark and held-out scenario runner for N+1 promotion.	Keep held-out cases separate from tuning. Save model/version/config and exact results for reproducibility.

Model selection workflow

1. Inspect the live Crusoe Managed Inference catalog and note exact Nemotron model IDs, if present.
2. Confirm context length, tool/function calling, structured output, streaming, concurrency, latency, and rate limits.
3. Run a fixed WIMBL evaluation covering multi-step planning, source-grounded synthesis, tool choice, verifier defect detection, and user-constraint adherence.
4. Compare quality and verified success per dollar, not only parameter count or advertised context length.
5. Pin the exact model ID and version in each run record so N versus N+1 comparisons remain interpretable.

Important: do not assume NVIDIA uses a particular model through Crusoe, or that every Nemotron model is available on Crusoe. The safe selection is a specific NVIDIA Nemotron model currently exposed by the required Crusoe route, with compatibility and performance confirmed. If none qualifies, check the hackathon rules or organizer before choosing another deployment path.

Official references

NVIDIA NemoClaw: https://www.nvidia.com/en-us/ai/nemoclaw

NVIDIA Nemotron: https://developer.nvidia.com/topics/ai/nemotron

Crusoe Managed Inference: confirm current endpoint, model catalog, and event-specific requirements in the official Crusoe documentation and hackathon materials.

6. Done Means Done: execution controller

Every run should have a bounded state machine. The controller, not the planner alone, decides whether to continue, replan, stop, or escalate.

Suggested lifecycle

```
REQUEST → INFER CONSTRAINTS → FREEZE ACCEPTANCE CRITERIA → DISCOVER → PLAN → VERIFY → (REPAIR / REPLAN)
→ FINAL RESULT → RETROSPECTIVE → N+1 EVALUATION
```

- Freeze required criteria at run start; do not silently relax them after failure.
- Set explicit retry, wall-clock, token, and tool-call budgets.
- Detect repeated tool calls, circular replanning, and no-progress loops.
- If the budget is exhausted, return a truthful partial result and explain what failed.
- Escalate only for a truly blocking ambiguity, authorization, or safety issue.
- Keep learning/promotion separate from the current trip's acceptance criteria.

7. Suggested repository layout

```
wimbl/
■■■ frontend/ # Traveler UI + agent operations + N+1 lab
■■■ backend/
■ ■■■ api/ # Requests, run state, results
■ ■■■ agents/ # Parent/child registry and specialists
■ ■■■ controller/ # Acceptance criteria, budgets, retries, stop rules
■ ■■■ memory/ # Scoped lessons, evidence, lineage
■ ■■■ improvement/ # Baseline, candidate, held-out eval, promotion, rollback
■ ■■■ integrations/ # You.com and Crusoe adapters
■■■ evals/
■ ■■■ benchmark_scenarios.json
■ ■■■ heldout_scenarios.json
■■■ tests/
■■■ .env.example
■■■ README.md
```

8. Three-minute demo plan

1. **0:00–0:25 | Goal:** enter a football-trip request. Show WIMBL inferring reasonable defaults instead of asking a questionnaire.
2. **0:25–1:05 | Family at work:** show ticket, flight, hotel, and verifier agents sharing findings with source URLs and timestamps.
3. **1:05–1:35 | Done Means Done:** verifier catches a real issue such as incompatible arrival timing or unsupported ticket claims; controller repairs or flags it.
4. **1:35–2:10 | Version N+1:** show a real lesson proposal, baseline versus candidate evaluation, held-out result, and promote/reject decision.
5. **2:10–2:45 | Inheritance:** run a second trip or controlled replay and show which proven lessons were inherited and what changed.
6. **2:45–3:00 | Evidence:** show actual metrics, exact model ID, inference route, and provenance. Never fabricate improvement numbers.

9. Definition of done for the MVP

- A natural-language request produces a ranked, source-backed travel package.
- Controller records constraints, assumptions, acceptance criteria, budgets, and run status.
- Verifier can fail a package and trigger bounded repair or replanning.
- At least one persistent parent lesson is inherited by a later trip agent.
- N+1 evaluates one isolated change against baseline and held-out scenarios, recording promotion or rejection.
- UI makes evidence, uncertainty, inherited lessons, and version comparisons visible.
- Exact Crusoe-served model ID and version are recorded, without unsupported claims about NVIDIA usage.
- No unnecessary questions are asked, while required authorization gates remain intact.

Implementation priority: first make one end-to-end trip verifiable; then add evidence-backed family memory; then ship the N+1 evaluation gate. Keep model and runtime behind adapters so the team can change them without rewriting the controller or learning engine.